

BITS, PILANI – K. K. BIRLA GOA CAMPUS

Theory of Computation

(CS F51)

Lecture CFG - 2



Context free grammar

A *context free grammar* (*cfd*) G is a quadruple (V, Σ, R, S) where

V is an (finite) alphabet set

$\Sigma \subseteq V$ is the set of *terminals*

S (the *start symbol*) is an element of $V - \Sigma$

R (the *set of rules*) is a (finite) subset of $(V - \Sigma) \times \Sigma^*$

The members of $V - \Sigma$ are called *non-terminals*

Notation:

Whenever $(A, u) \in R$ we write $A \rightarrow_G u$

Context free language

For any $u, v \in \Sigma^*$, we write $u \Rightarrow_G v$ if and only if there are strings $x, y \in V^*$ & $A \in V - \Sigma$ such that $u = xAy$, $v = xv'y$ and $A \rightarrow_G v'$

The relation $\Rightarrow_G^*$ is the reflexive transitive closure of $\Rightarrow_G$

$L(G)$ the language generated by G is $\{w \in \Sigma^* : S \Rightarrow_G^* w\}$

A language L is said to be a *context free language* (*cfl*) if $L = L(G)$ for some *context free grammar* G .

We call any sequence of the form

$$w_0 \Rightarrow_G w_1 \Rightarrow_G \dots \Rightarrow_G w_n$$

a derivation in G of w_n from w_0 , where $w_i \in V^*$

We also say that the derivation has n steps

Context free grammar

Example: $L = \{a^n b^n : n \geq 0\}$

Consider $G = (V, \Sigma, R, S)$ where

$V = \{S, a, b\}$

$\Sigma = \{a, b\}$

R consists of the rules $S \rightarrow e$, $S \rightarrow aSb$

Consider a^2b^2 , then the possible derivation is

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$

The **1st two steps** used the rule $S \rightarrow aSb$

and the **last step** used the rule $S \rightarrow e$

We also write $S \rightarrow e \mid aSb$

Context free grammar

Example: $L = \{a^n b^n : n \geq 0\} \cup \{b^n a^n : n \geq 0\}$

Let $L_1 = \{a^n b^n : n \geq 0\}$ and $L_2 = \{b^n a^n : n \geq 0\}$

Then $L = L_1 \cup L_2$

We know L_1 and L_2 are cfl's

Consider $G = (V, \Sigma, R, S)$ where

$V = \{S, S_1, S_2, a, b\}$, $\Sigma = \{a, b\}$

R consists of the rules

$S \rightarrow S_1$, $S \rightarrow S_2$

$S_1 \rightarrow e$, $S_1 \rightarrow a S_1 b$

$S_2 \rightarrow e$, $S_2 \rightarrow b S_2 a$

CFL

Example

Language of all strings of properly balanced left and right parentheses, like $()(())$

We let $G = (V, \Sigma, R, S)$

where $V = \{S, (,)\}$, $\Sigma = \{ (,) \}$

R consists of the rules

$$S \rightarrow e, S \rightarrow SS, S \rightarrow (S)$$

Derivation of $()(())$

$$S \Rightarrow SS \Rightarrow S(S) \Rightarrow S((S)) \Rightarrow S(()) \Rightarrow (S)(()) \Rightarrow ()(())$$

Context free grammar

Example: $L = \{a, b\}^*$

Think S as a variable representing arbitrary element of L .

Note:

$e \in L$ and if $w \in L$, then $wa \in L$ & $wb \in L$

Consider $G = (V, \Sigma, R, S)$ where

$V = \{S, a, b\}$

$\Sigma = \{a, b\}$

R consists of the rules $S \rightarrow e, S \rightarrow Sa, S \rightarrow Sb$

Is this the only way to define/generate L ?

Think : $x \in L$ & $y \in L$ then $xy \in L$

$S \rightarrow e, S \rightarrow a, S \rightarrow b, S \rightarrow SS$

Context free grammar

Example: L = language of *palindromes* over $\{a, b\}$

Note:

- $e \in L$ and $a, b \in L$
- any palindrome must start and end with same symbol.
- For any $S \in L$, $aSa \in L$ and $bSb \in L$

Consider

$S \rightarrow e \mid a \mid b \mid aSa \mid bSb$

Next, consider the *complement of L* ;

L' = language of *non-palindromes* over $\{a, b\}$

For *regular languages* we know that *complement* of a regular language is regular.

What about cfl? In general, answer is **NO**

Context free grammar

How to generate all possible *non-palindromes*

- For any $S \in L'$, $aSa \in L'$ and $bSb \in L'$

$$S \rightarrow aSa \mid bSb$$

- For every $A \in \{a, b\}^*$, $aAb \in L'$ and $bAa \in L'$

$$S \rightarrow aAb \mid bAa$$

$$A \rightarrow Aa \mid Ab \mid e$$

CFL

Note: Every regular language is an context free language

Let L be a regular language and M be the DFA such that $L = L(M)$.

Let $M = (K, \Sigma, \delta, s, F)$.

Define the grammar $G = (V, \Sigma, R, S)$ as follows:

$V = K \cup \Sigma$, $S = s$ and R consists of the rules :

$R = \{q \rightarrow ap : \delta(q, a) = p\} \cup \{q \rightarrow e : q \in F\}$

The non-terminals are the states of the automata;

For each transition from q to p on input a we have in R the rule $q \rightarrow ap$.

Parse Trees

Consider the CFG for the properly balanced left & right parentheses

$S \rightarrow e, S \rightarrow SS, S \rightarrow (S)$

Consider the string $() ()$

This can be derived as

$S \Rightarrow SS \Rightarrow (S)S \Rightarrow ()S \Rightarrow ()(S) \Rightarrow () ()$

It can also be derived as

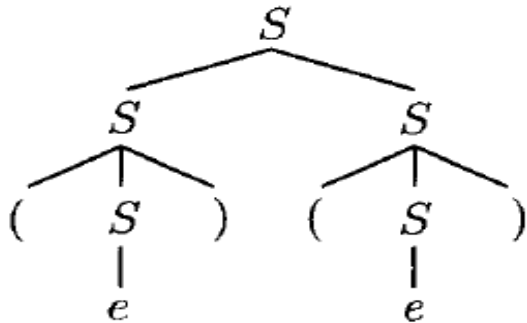
$S \Rightarrow SS \Rightarrow S(S) \Rightarrow S() \Rightarrow (S)() \Rightarrow () ()$

Observe:

- In these two derivations, *the rules applied at each step are same*
- *Except* the *difference is in the order in which the rules are applied.*

Parse Trees

We can view both derivations as



We call such a view as a *parse tree*.

The points are called *nodes*.

The topmost node is called the *root*.

The nodes along the bottom are called *leaves*.

By *concatenating* the labels of the *leaves* from left to right, we obtain the derived string, which is called the *yield* of the parse tree

Parse Trees

Formally, for an arbitrary CFG $G = \{V, \Sigma, R, S\}$, we define its *parse trees* and their *roots*, *leaves*, and *yields*, as follows:

1. $\circ a$

This is a *parse tree* for each $a \in \Sigma$. The single node of this parse tree is the *root* and a *leaf*. The *yield* of this parse tree is a .

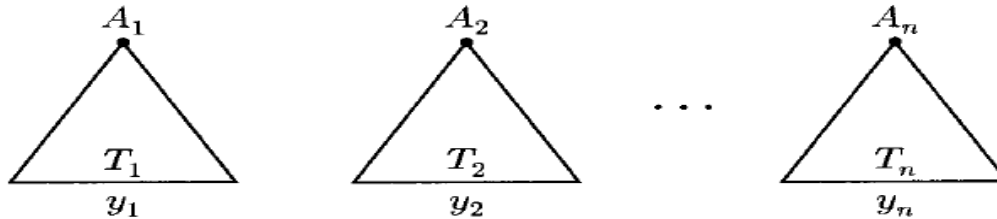
2. If $A \rightarrow e$ is a rule in R , then



is a *parse tree*; its *root* is the node labeled A , its sole *leaf* is the node labeled e , and its *yield* is e .

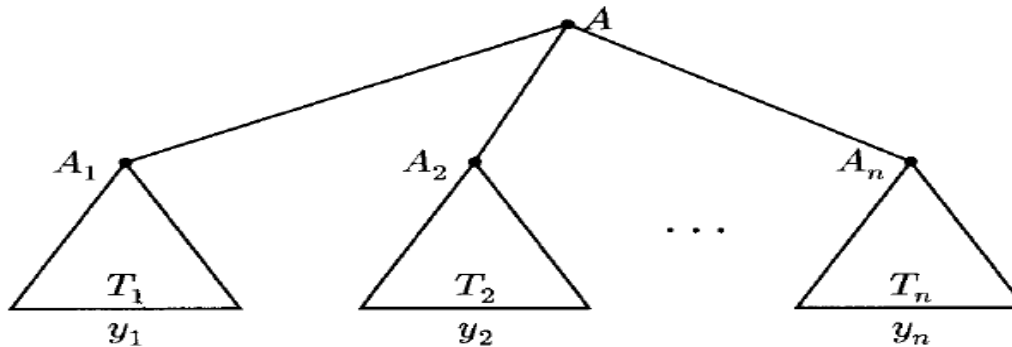
Parse Trees

3. If



are *parse trees*, where $n \geq 1$, with *roots* labeled

$A_1, A_2, \dots, A_n$ respectively and with *yields* $y_1, y_2, \dots, y_n$ and $A \rightarrow A_1 A_2 \dots A_n$ is a *rule* in R , then



is a *parse tree*. Its *root* is the new node labeled A , its *leaves* are the leaves of $T_1, T_2, \dots, T_n$ and the *yield* is

$y_1 y_2 \dots y_n$

Parse Trees

Note

Parse trees are ways of representing derivations so that the *superficial differences* between derivations (because of order of applications of rules) are suppressed.

Parse Tree represent *equivalent derivations*.

Let $G = \{V, \Sigma, R, S\}$ be a CFG.

Let $D : x_1 \Rightarrow x_2 \Rightarrow \dots \Rightarrow x_n$

and $D' : x'_1 \Rightarrow x'_2 \Rightarrow \dots \Rightarrow x'_n$

be *two derivations* in G , where $x_i, x'_i \in V^*$ for $i=1,2,\dots,n$,
 $x_1, x'_1 \in V - \Sigma$ and $x_n, x'_n \in \Sigma^*$

Equivalent Derivations

We say that D *precedes* D' denoted $D < D'$ if $n > 2$ and there is an integer k , $1 < k < n$ such that

1. For all $i \neq k$ we have $x_i = x'_i$
2. $x_{k-1} = x'_{k-1} = uAvBw$ where $u, v, w \in V^*$ and $A, B \in V - \Sigma$
3. $x_k = uyvBw$, where $A \rightarrow y \in R$
4. $x'_k = uAvzw$, where $B \rightarrow z \in R$
5. $x_{k+1} = x'_{k+1} = uyvzw$

Example: Consider the three derivations D_1, D_2, D_3 then

$$D_1 = S \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow (())S \Rightarrow (())(S) \Rightarrow (())()$$

$$D_2 = S \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow ((S))(S) \Rightarrow (())(S) \Rightarrow (())()$$

$$D_3 = S \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow ((S))(S) \Rightarrow ((S))() \Rightarrow (())()$$

Then $D_1 < D_2$
and $D_2 < D_3$
but D_1 *does not precede* D_3
We say D_1 and D_3 are *similar*

Equivalent Derivations

We say two derivations D and D' are *similar* if they belong in the reflexive, transitive closure of $<$

Leftmost Derivation

Repeatedly *replace* the *leftmost nonterminal* in the *current string* according to the rule suggested by the parse tree.

It is *not preceded* by any other derivation.

Rightmost Derivation

Repeatedly *replace* the *rightmost nonterminal* in the *current string* according to the rule suggested by the parse tree.

It *does not precede* any other derivation.